

SVM - Practical Implementation

Virtual Labs - Dayalbagh Educational Institute

Step 1: Importing Libraries

Importing Libraries

```
# Numerical and data handling libraries
import numpy as np
import pandas as pd

# Visualization libraries
import matplotlib.pyplot as plt
import seaborn as sns
from mlxtend.plotting import plot_decision_regions

print("Loaded libraries")
```

Output:

Loaded libraries

Dataset utilities

```
from sklearn.datasets import load_wine, make_moons
# Preprocessing and model utilities
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Models
from sklearn.svm import SVC
# Evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

print("Loaded Sklearn libraries")
```

Output:

Loaded Sklearn libraries

Step 2: Loading Dataset

Loading and Reading Data

```
wine = load_wine()
wine_df = pd.DataFrame(wine.data, columns=wine.feature_names)

wine_df["target"] = wine.target
wine_df
```

Output:

	alco.	malic_a.	ash	alcalinity_of.	magnes.	total_phe.	flavano.	nonflavanoid_phe.	proanthocya.	color_inten.	hue	od280/od315_of_diluted_w.	prol.	tar.
0	14.23	1.71	2.4	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.0	3.92	1065.0	0
1	13.20	1.78	2.4	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.0	3.40	1050.0	0
2	13.16	2.36	2.4	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.0	3.17	1185.0	0
3	14.37	1.95	2.4	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.8	3.45	1480.0	0
4	13.24	2.59	2.4	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.0	2.93	735.0	0
...														
175	13.71	5.65	2.4	20.5	95.0	1.68	0.61	0.52	1.06	7.70	0.6	1.74	740.0	2
176	13.40	3.91	2.4	23.0	102.0	1.80	0.75	0.43	1.41	7.30	0.7	1.56	750.0	2
177	13.27	4.28	2.4	20.0	120.0	1.59	0.69	0.43	1.35	10.20	0.5	1.56	835.0	2
178	13.17	2.59	2.4	20.0	120.0	1.65	0.68	0.53	1.46	9.30	0.6	1.62	840.0	2
179	14.13	4.10	2.4	24.5	96.0	2.05	0.76	0.56	1.35	9.20	0.6	1.60	560.0	2

Output:

178 rows x 14 columns

Display Wine dataset info

```
wine_df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   alcohol                               178 non-null    float64
1   malic_acid                            178 non-null    float64
2   ash                                   178 non-null    float64
3   alcalinity_of_ash                     178 non-null    float64
4   magnesium                             178 non-null    float64
5   total_phenols                         178 non-null    float64
6   flavanoids                            178 non-null    float64
7   nonflavanoid_phenols                  178 non-null    float64
8   proanthocyanins                       178 non-null    float64
9   color_intensity                       178 non-null    float64
10  hue                                   178 non-null    float64
11  od280/od315_of_diluted_wines          178 non-null    float64
12  proline                               178 non-null    float64
13  target                                178 non-null    int32
dtypes: float64(13), int32(1)
memory usage: 19.0 KB
```

Generate two-moons dataset

```
X_moons, y_moons = make_moons(n_samples=700, noise=0.15, random_state=42)
moons_df = pd.DataFrame(X_moons, columns=["Feature_1", "Feature_2"])
moons_df["target"] = y_moons
moons_df
```

Output:

	Feature_1	Feature_2	target
0	-0.036777	1.017428	0
1	0.991181	-0.542893	1
2	0.135605	0.095587	1
3	0.380490	0.882966	0
4	-0.795374	0.193136	0
...			
695	1.147087	0.520498	0
696	0.200663	0.598349	0
697	-0.384952	0.547244	0
698	0.452570	-0.174473	1
699	0.534527	0.910810	0

Output:

700 rows x 3 columns

Step 3: Data Analysis

Data Analysis

```
wine_df = pd.concat(
    [wine_df[["flavanoids", "color_intensity"]], wine_df["target"]],
    axis=1
)

wine_target_names = {
    0: "Barolo",
    1: "Grignolino",
    2: "Barbera"
}

wine_df["class_name"] = wine_df["target"].map(wine_target_names)
wine_df[["target", "class_name"]].value_counts()
```

Output:

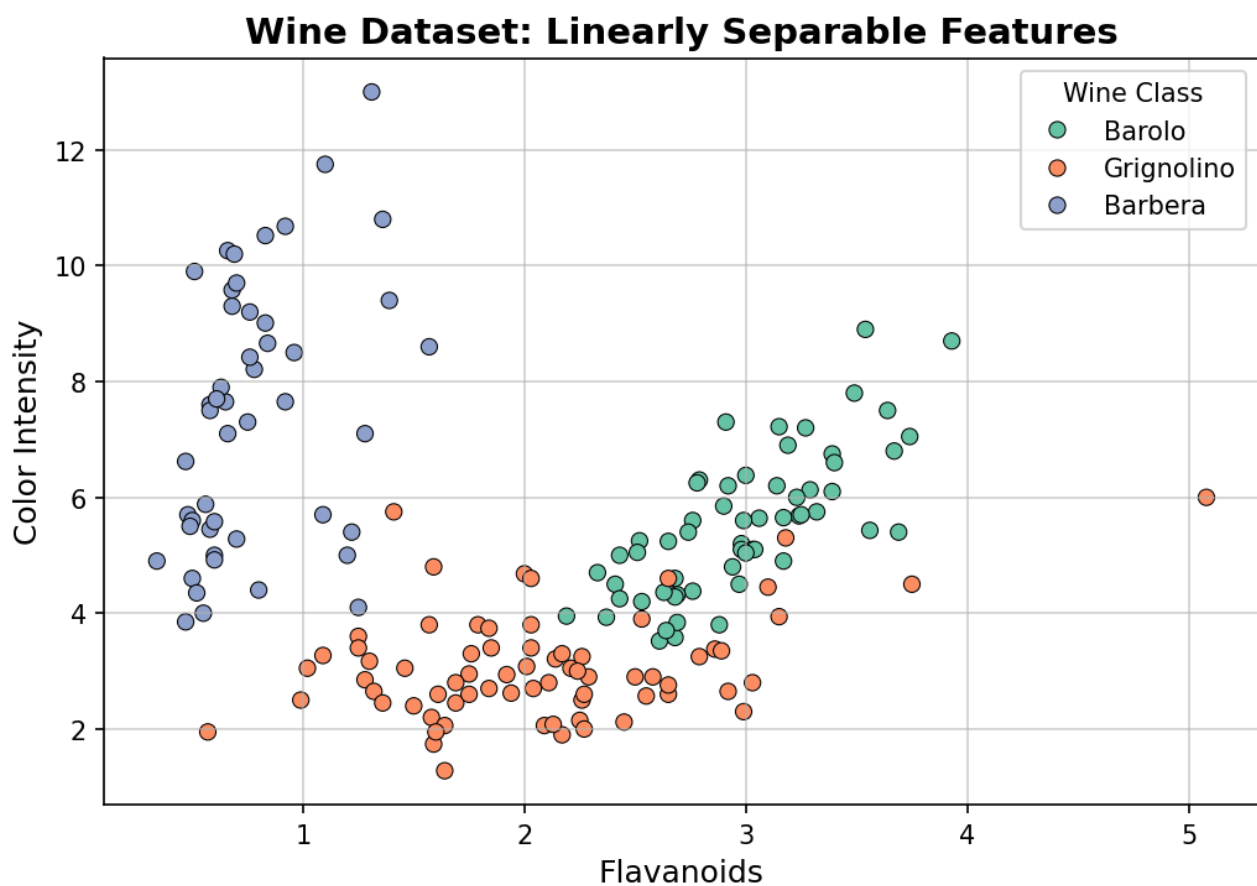
```
target  class_name
1      Grignolino    71
0      Barolo       59
2      Barbera      48
Name: count, dtype: int64
```

Visualizing Wine Dataset

```

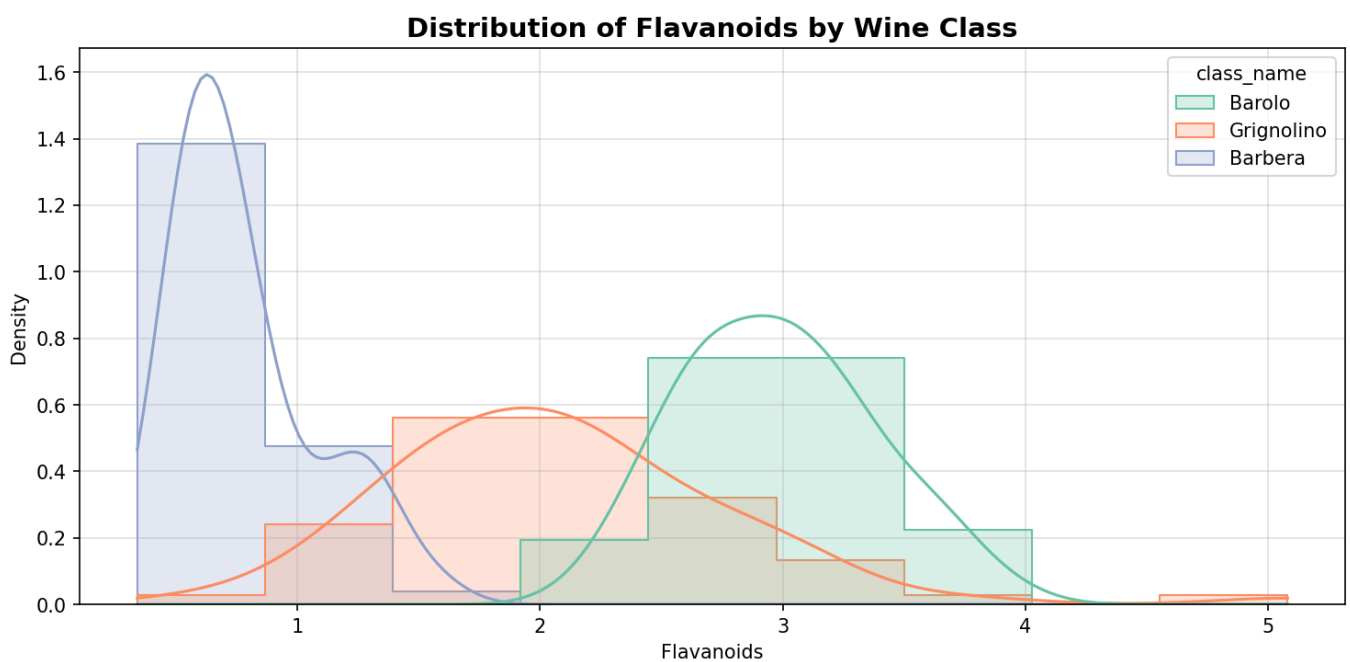
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=wine_df,
    x="flavanoids",
    y="color_intensity",
    hue="class_name",
    palette="Set2",
    s=40,
    edgecolor="black"
)
plt.title("Wine Dataset: Linearly Separable Features", fontsize=14, fontweight="bold")
plt.xlabel("Flavanoids", fontsize=12)
plt.ylabel("Color Intensity", fontsize=12)
plt.legend(title="Wine Class")
plt.grid(alpha=0.6)
plt.show()

```



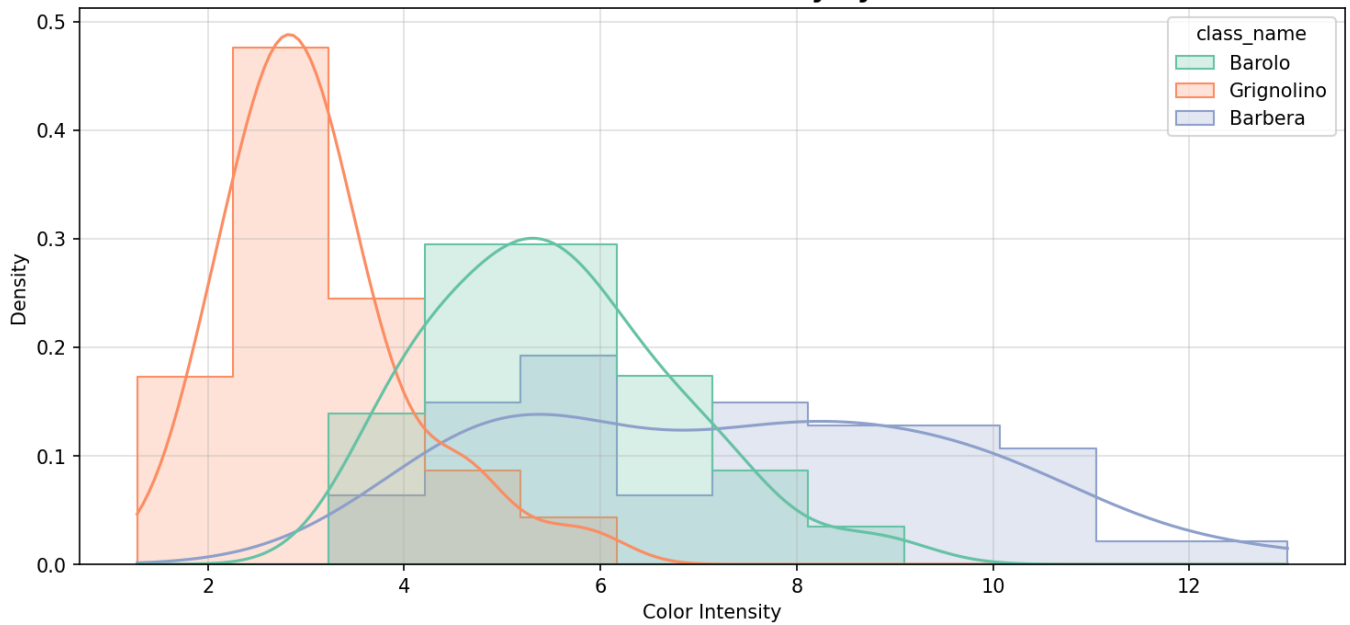
Histogram of Flavanoids

```
plt.figure(figsize=(10, 5))
sns.histplot(
    data=wine_df,
    x="flavanoids",
    hue="class_name",
    kde=True,
    palette="Set2",
    element="step",
    stat="density",
    common_norm=False
)
plt.title("Distribution of Flavanoids by Wine Class", fontsize=14, fontweight="bold")
plt.xlabel("Flavanoids")
plt.ylabel("Density")
plt.grid(alpha=0.4)
plt.show()
```

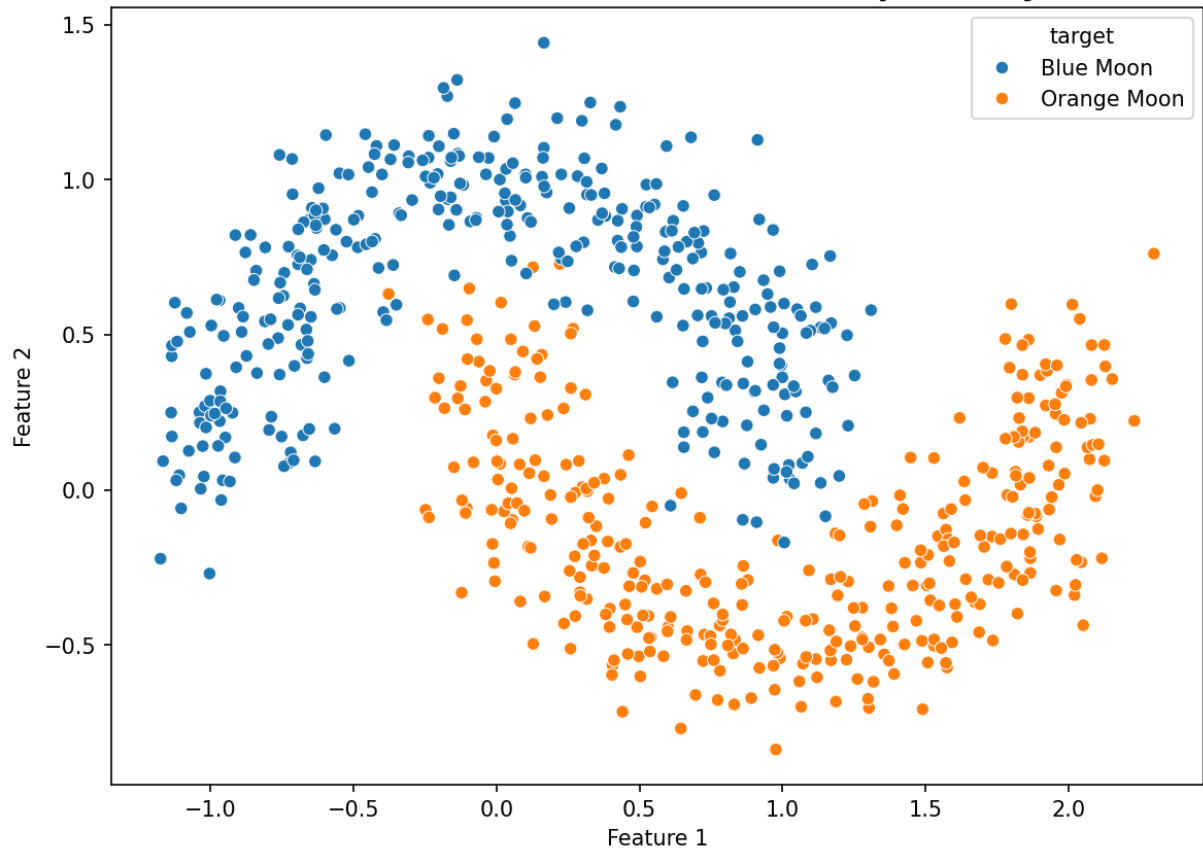


Histogram of Color Intensity

```
plt.figure(figsize=(10, 5))
sns.histplot(
    data=wine_df,
    x="color_intensity",
    hue="class_name",
    kde=True,
    palette="Set2",
    element="step",
    stat="density",
    common_norm=False
)
plt.title("Distribution of Color Intensity by Wine Class", fontsize=14, fontweight="bold")
plt.xlabel("Color Intensity")
plt.ylabel("Density")
plt.grid(alpha=0.4)
plt.show()
```

Distribution of Color Intensity by Wine Class**# Visualizing Two Moons Dataset**

```
plt.figure(figsize=(8,6))
sns.scatterplot(
    x=moons_df["Feature_1"],
    y=moons_df["Feature_2"],
    hue=moons_df["target"].map({0: "Blue Moon", 1: "Orange Moon"}),
    palette=["#1f77b4", "#ff7f0e"]
)
plt.title("Two Moons Dataset - Non-Linear Separability", fontsize=14, fontweight='bold')
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Two Moons Dataset - Non-Linear Separability**Step 4: Data Preprocessing****# Data Preprocessing**

```
X_wine = wine_df[["flavanoids", "color_intensity"]]
y_wine = wine_df["target"]

Xw_train, Xw_test, yw_train, yw_test = train_test_split(
    X_wine, y_wine, test_size=0.2, random_state=42, stratify=y_wine
)
Xw_train.shape, Xw_test.shape, yw_train.shape, yw_test.shape
```

Output:

```
((142, 2), (36, 2), (142,), (36,))
```

Train/Test split for Two Moons dataset

```
Xm_train, Xm_test, ym_train, ym_test = train_test_split(
    X_moons, y_moons, test_size=0.2, random_state=42, stratify=y_moons
)
Xm_train.shape, Xm_test.shape, ym_train.shape, ym_test.shape
```

Output:

```
((560, 2), (140, 2), (560,), (140,))
```

Feature Scaling

```
scaler = StandardScaler()
scaler
```

Output:
StandardScaler()

Scale Wine training and test data

```
Xw_train_scaled = scaler.fit_transform(Xw_train)
Xw_test_scaled = scaler.transform(Xw_test)

print("Wine data scaled successfully!")
```

Output:
Wine data scaled successfully!

View original Wine training data (before scaling)

```
Xw_train[:5]
```

Output:

	flavanoids	color_intensity
10	2.81	5.40
149	3.17	5.75
90	3.00	5.50
59	2.91	5.60
147	0.66	9.39

View scaled Wine training data (after scaling)

```
Xw_train_scaled[:5]
```

Output:
array([[0.73229212, -0.16746725],
[1.33318146, 0.30530313],
[1.006382 , -0.081509],
[0.81662747, 0.262324],
[-1.29175618, 1.47433535]])

Scale Two Moons training and test data

```
Xm_train_scaled = scaler.fit_transform(Xm_train)
Xm_test_scaled = scaler.transform(Xm_test)

print("Two Moons data scaled successfully!")
```

Output:
Two Moons data scaled successfully!

View original Two Moons training data (before scaling)

```
Xm_train[:5]
```

Output:

```
array([[ 1.3723603, -0.55081882],
       [ 0.63613117,  0.78345978],
       [-0.43531115,  0.96004336],
       [ 0.41718644,  1.17721089],
       [-1.13531808,  0.17220389]])
```

View scaled Two Moons training data (after scaling)

```
Xm_train_scaled[:5]
```

Output:

```
array([[ 1.01466867, -1.57697415],
       [ 0.17320777,  1.01862443],
       [-1.05137949,  1.36213595],
       [-0.07703149,  1.78459625],
       [-1.85144083, -0.17046376]])
```

Step 5: Model Training

Model Training

```
# Traing linear Kernel SVM in wine dataset with 2 features
svm_linear_wine = SVC(kernel="linear")
svm_linear_wine.fit(Xw_train_scaled, yw_train)
svm_linear_wine
```

Output:

```
SVC(kernel='linear')
```

Training SVMs with linear and RBF kernels on two-moons toy dataset

```
svm_linear_moon = SVC(kernel="linear")
svm_rbf_moon = SVC(kernel="rbf")

svm_linear_moon.fit(Xm_train_scaled, ym_train)
svm_rbf_moon.fit(Xm_train_scaled, ym_train)
print("Model trained")
```

Output:

```
Model trained
```

Step 6: Model Evaluation

Model Evaluation

```
# Wine dataset case
wine_preds = svm_linear_wine.predict(Xw_test_scaled)
wine_preds
```

Output:

```
array([0, 2, 0, 0, 1, 0, 0, 0, 1, 2, 1, 2, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1,
       0, 0, 0, 1, 0, 2, 1, 2, 0, 2, 1, 2, 2, 2])
```

Wine dataset - Classification metrics

```
print("Accuracy:", accuracy_score(yw_test, wine_preds))
print("Precision:", precision_score(yw_test, wine_preds, average="macro"))
print("Recall:", recall_score(yw_test, wine_preds, average="macro"))
print("F1 Score:", f1_score(yw_test, wine_preds, average="macro"))

print("\n\nClassification Report:\n\n")
print(classification_report(yw_test, wine_preds))
```

Output:

```
Accuracy: 0.8611111111111112
Precision: 0.8772893772893773
Recall: 0.8674603174603175
F1 Score: 0.8694456940070975
Classification Report:
precision    recall  f1-score   support

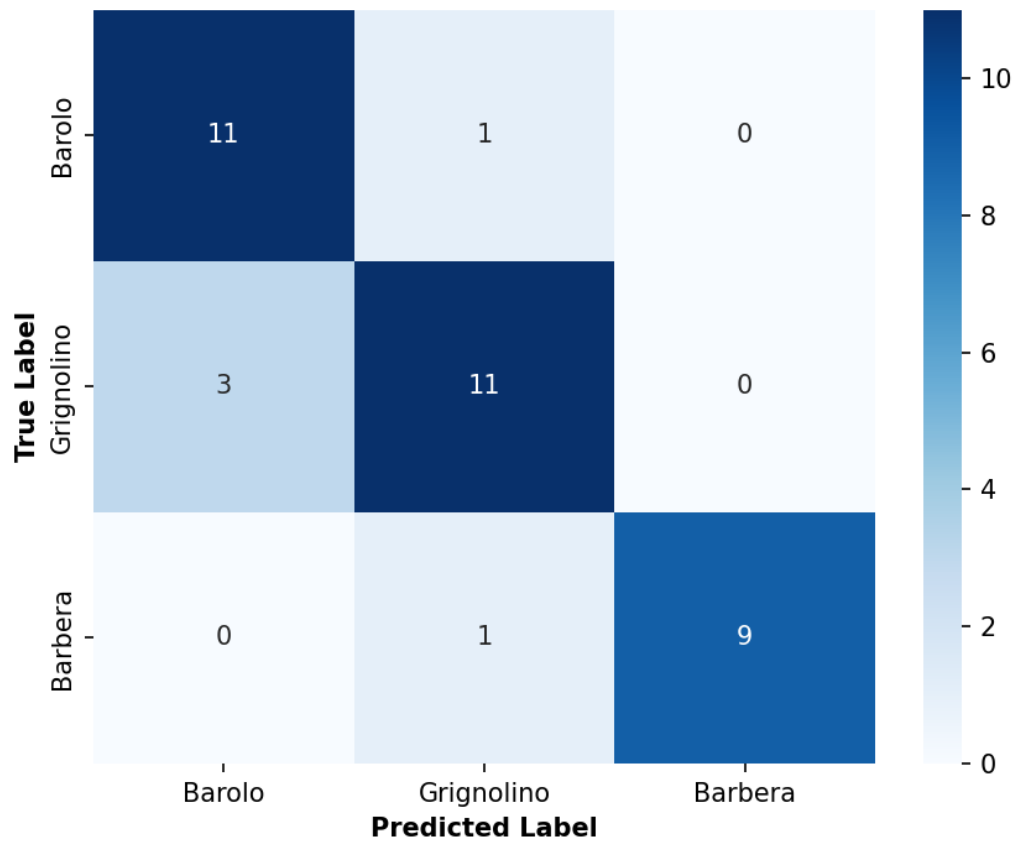
0           0.79      0.92      0.85         12
1           0.85      0.79      0.81         14
2           1.00      0.90      0.95         10
accuracy                   0.86         36
macro avg           0.88      0.87      0.87         36
weighted avg           0.87      0.86      0.86         36
```

Confusion Matrix

```
labels = list(wine_target_names.keys())
class_names = list(wine_target_names.values())

plt.figure(figsize=(6, 5))
sns.heatmap(
    confusion_matrix(yw_test, wine_preds, labels=labels),
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Wine Dataset (Linear Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()
```

Wine Dataset (Linear Kernel)**# Two moon dataset case**

```
moon_linear_preds = svm_linear_moon.predict(Xm_test_scaled)
moon_linear_preds
```

Output:

```
array([0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1,
0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0,
1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0,
0, 0, 1, 0, 0, 1, 1, 1])
```

Confusion Matrix

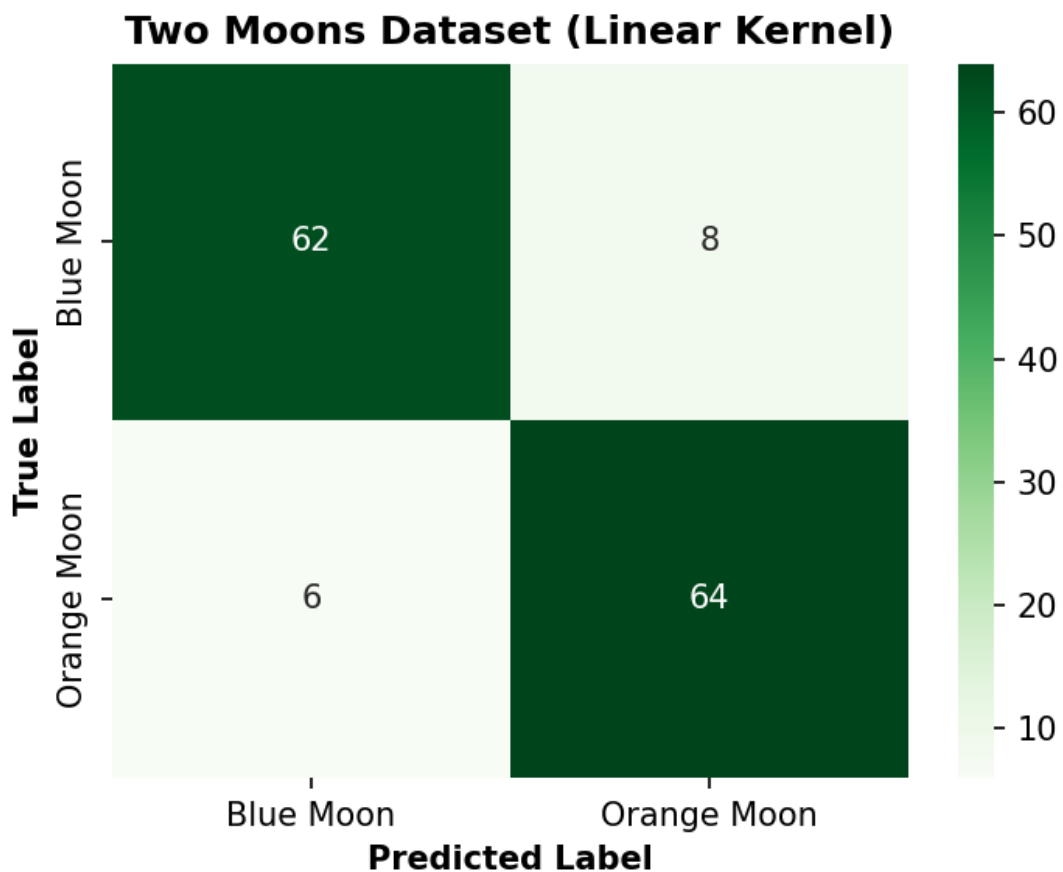
```

class_names = ['Blue Moon', 'Orange Moon']

plt.figure(figsize=(5, 4))
sns.heatmap(
    confusion_matrix(ym_test, moon_linear_preds),
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Two Moons Dataset (Linear Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()

```



Two Moons - Linear kernel classification metrics

```

print("Accuracy:", accuracy_score(ym_test, moon_linear_preds))
print("Precision:", precision_score(ym_test, moon_linear_preds, average="macro"))
print("Recall:", recall_score(ym_test, moon_linear_preds, average="macro"))
print("F1 Score:", f1_score(ym_test, moon_linear_preds, average="macro"))

print("\n\nClassification Report:\n\n")
print(classification_report(ym_test, moon_linear_preds))

```

Output:

```

Accuracy: 0.9
Precision: 0.9003267973856208
Recall: 0.8999999999999999
F1 Score: 0.8999795876709533
Classification Report:
precision    recall  f1-score   support
0           0.91      0.89      0.90         70
1           0.89      0.91      0.90         70
accuracy                  0.90        140
macro avg              0.90      0.90      0.90        140
weighted avg           0.90      0.90      0.90        140

```

Two moon dataset case

```

moon_rbf_preds = svm_rbf_moon.predict(Xm_test_scaled)
moon_rbf_preds

```

Output:

```

array([0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1,
0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0,
1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0,
1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0,
1, 1, 1, 0, 1, 1, 0, 1])

```

Two Moons - RBF Kernel Confusion Matrix

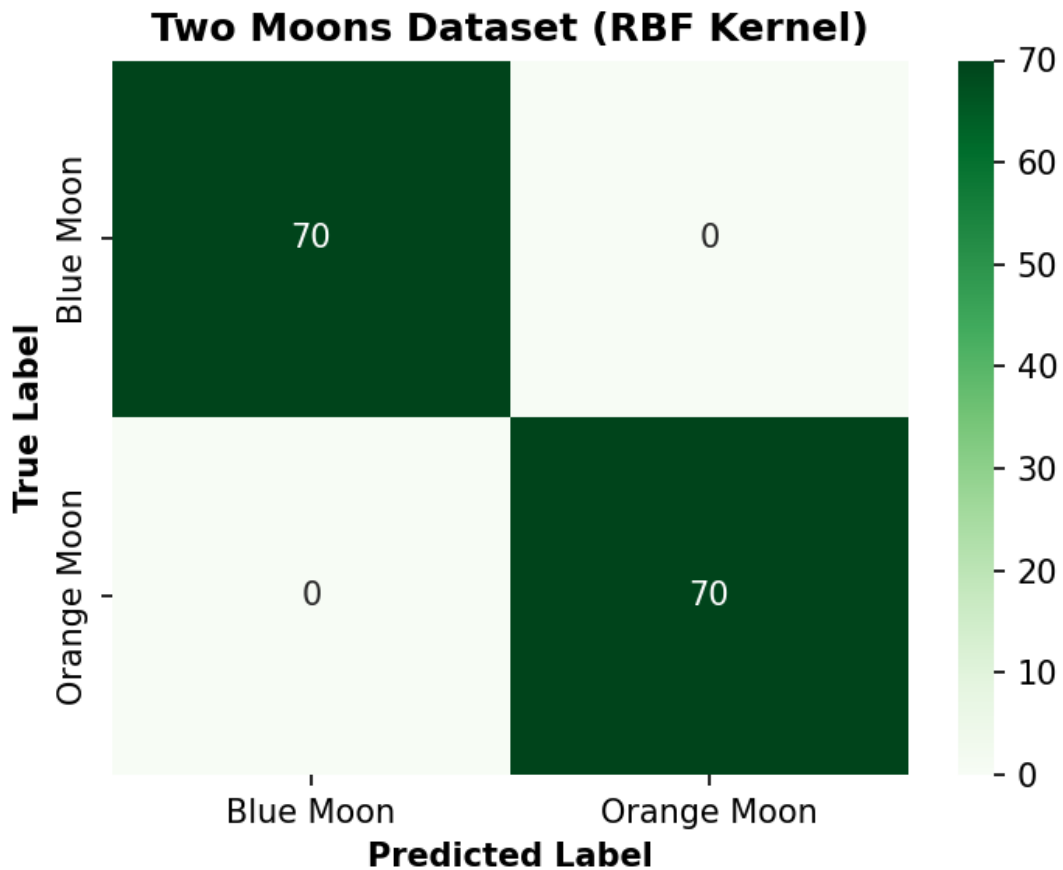
```

class_names = ['Blue Moon', 'Orange Moon']

plt.figure(figsize=(5, 4))
sns.heatmap(
    confusion_matrix(ym_test, moon_rbf_preds),
    annot=True,
    fmt="d",
    cmap="Greens",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.title("Two Moons Dataset (RBF Kernel)", fontweight='bold')
plt.xlabel("Predicted Label", fontweight='bold')
plt.ylabel("True Label", fontweight='bold')
plt.tight_layout()
plt.show()

```



Two Moons - RBF kernel classification metrics (Perfect accuracy!)

```
print("Accuracy:", accuracy_score(ym_test, moon_rbf_preds))
print("Precision:", precision_score(ym_test, moon_rbf_preds, average="macro"))
print("Recall:", recall_score(ym_test, moon_rbf_preds, average="macro"))
print("F1 Score:", f1_score(ym_test, moon_rbf_preds, average="macro"))

print("\n\nClassification Report:\n\n")
print(classification_report(ym_test, moon_rbf_preds))
```

Output:

```
Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
Classification Report:
precision    recall  f1-score   support

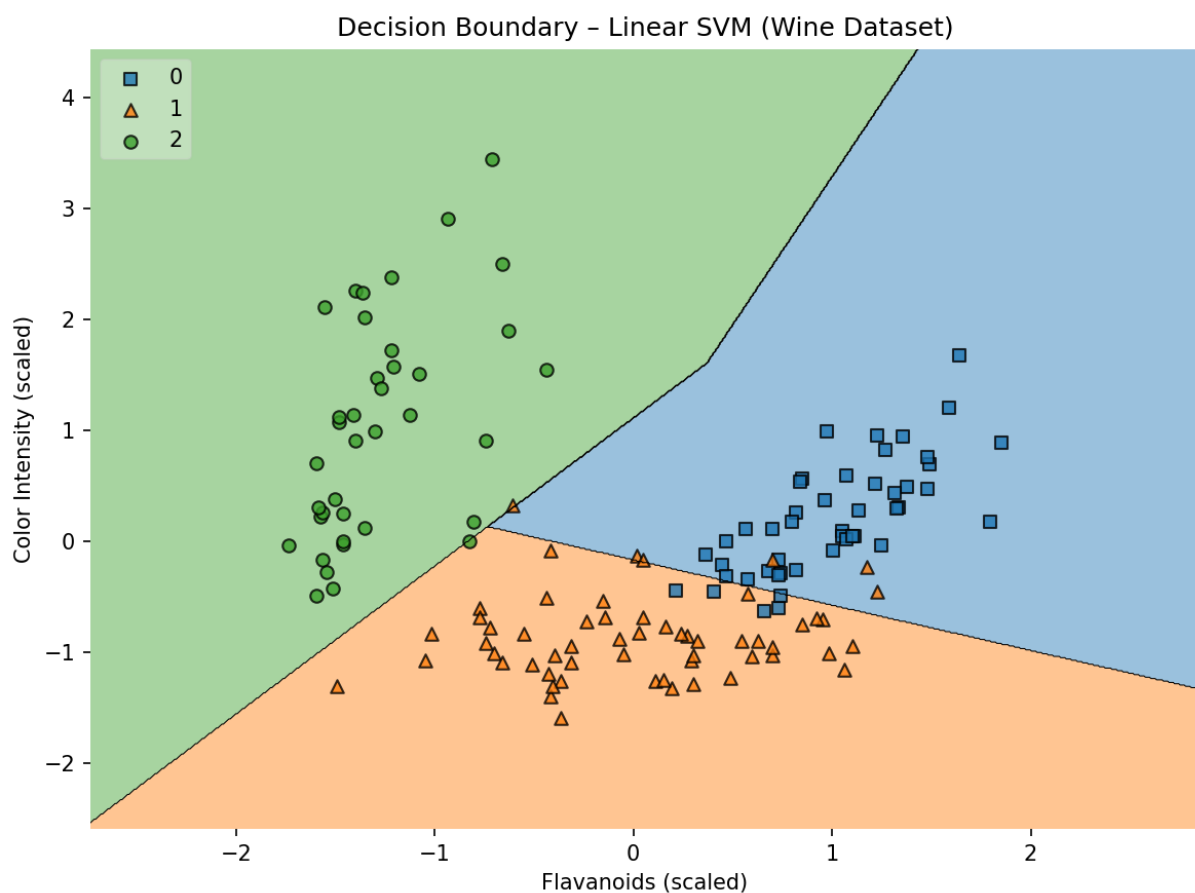
0           1.00      1.00      1.00        70
1           1.00      1.00      1.00        70

accuracy               1.00        140
macro avg              1.00      1.00      1.00        140
weighted avg           1.00      1.00      1.00        140
```

Step 7: Model Simulation

Model testing

```
# Decision Boundaries - Wine
plot_decision_regions(
    Xw_train_scaled,
    yw_train.values,
    clf=svm_linear_wine,
    legend=2
)
plt.title("Decision Boundary - Linear SVM (Wine Dataset)")
plt.xlabel("Flavanoids (scaled)")
plt.ylabel("Color Intensity (scaled)")
plt.show()
```



Decision Boundaries - Two Moons

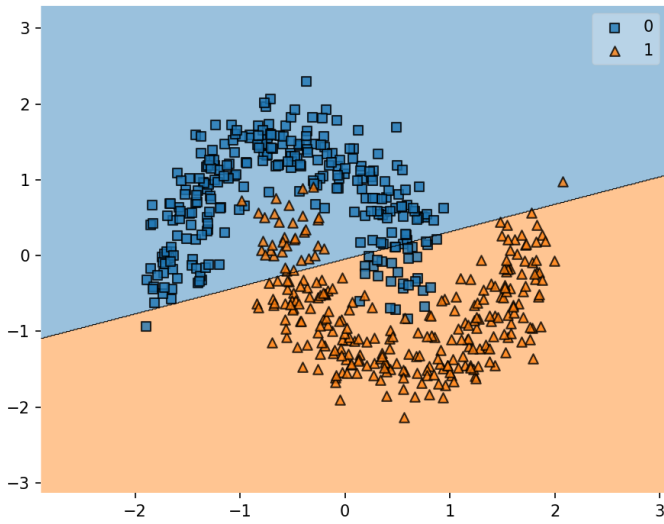
```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plot_decision_regions(Xm_train_scaled, ym_train, clf=svm_linear_moon)
plt.title("Linear Kernel - Blue vs Orange Moon")

plt.subplot(1,2,2)
plot_decision_regions(Xm_train_scaled, ym_train, clf=svm_rbf_moon)
plt.title("RBF Kernel - Blue vs Orange Moon")

plt.show()
```

Linear Kernel - Blue vs Orange Moon



RBF Kernel - Blue vs Orange Moon

